

Highlight | July 28, 2021

Computer Vision, Privacy

# Recognizing People in Photos Through Private On-Device Machine Learning

---



Photos (on iOS, iPad OS, and Mac OS) is an integral way for people to browse, search, and relive life's moments with their friends and family. Photos uses a number of machine learning algorithms, running privately on-device, to help curate and organize images, Live Photos, and videos. An algorithm foundational to this goal recognizes people from their visual appearance.

Photos relies on identity information in a number of ways. As shown in [Figure 1A](#), a user can scroll up on an image, tap on the circle representing the person that has been recognized in that image, and then pivot to browse their library to see images containing that person. A user can also directly access the People Album, shown in [Figure 1B](#), to browse images and confirm the correct person is tagged in their images. A user can then manually add names to people in their photos and find someone by typing the person's name in the search bar, as shown in [Figure 1C](#).

Photos can also learn from identity information to build a private, on-device knowledge graph that identifies interesting patterns in a user's library, such as important groups of people, frequent places, past trips, events, the last time a user took an image of a certain person, and more. The knowledge graph powers the beloved Memories feature in Photos, which creates engaging video vignettes centered around different themes in a user's library. Memories uses popular themes based on important people in a user's life, such as a memory for "Together," as shown in [Figure 1D](#).



Figure 1: A) Picture showing the identified people at the bottom left. B) People Album. C) Compound persons query search. D) Memories pane showing person-specific memories.

The task of recognizing people in user-generated content is inherently challenging because of the sheer variability in the domain. People can appear at arbitrary scales, lighting, pose, and expression, and the images can be captured from any camera. When someone wants to view all their photos of a specific person, a comprehensive knowledge graph is needed, including instances where the subject is not posing for the image. This is especially true in photography of dynamic scenes, such as capturing a toddler bursting a bubble, or friends raising a glass for a toast.

Another challenge, and a fundamental requirement for automatic person recognition, is to ensure equity in the results. People all around the world use Apple products. We want everyone to have the same extraordinary experience that we designed into the feature, no matter the photographic subject's skin color, age, or gender.

## Recognizing People in Image Collections

Recognizing people in libraries consists of two interwoven phases. One phase involves constructing a gallery of known individuals progressively as the library evolves. The second phase consists of assigning a new person observation to either a known individual in the gallery or declaring the observation as an unknown individual. The algorithms in both of these phases operate on feature vectors, also called embeddings, that represent a person observation.

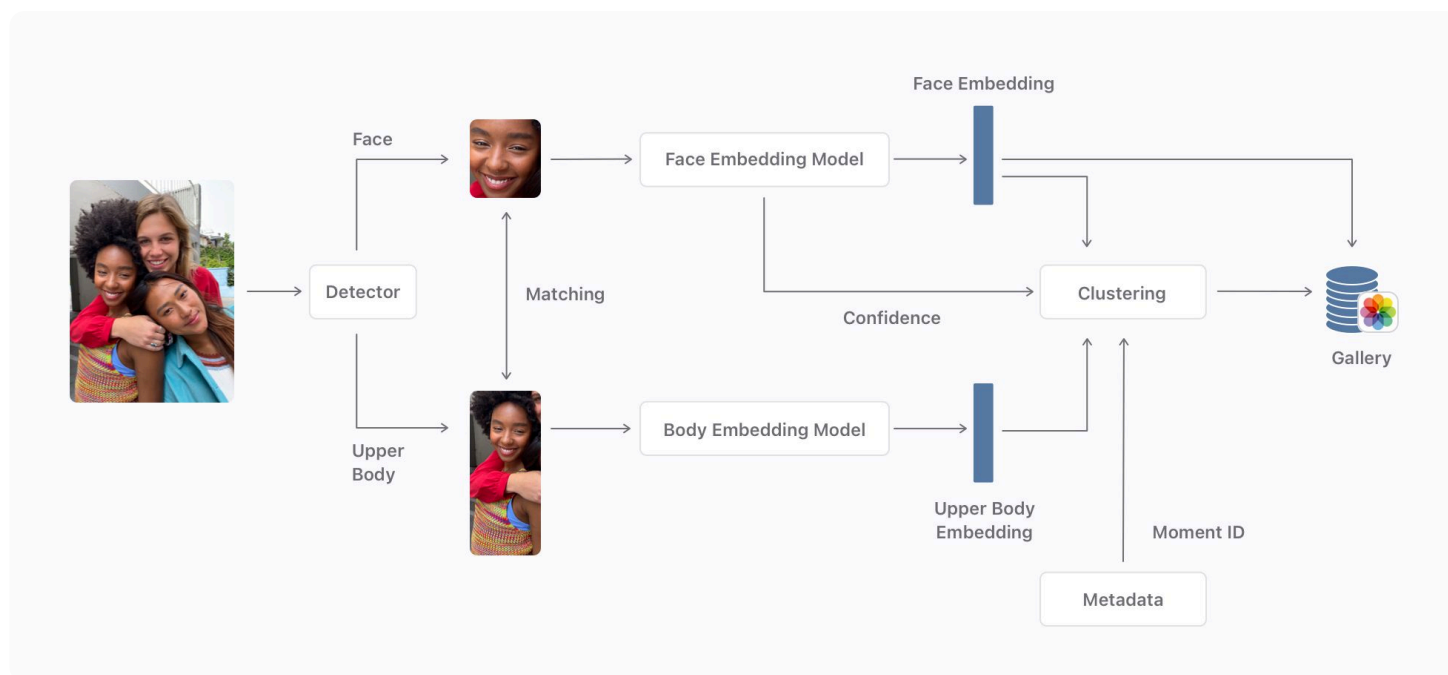


Figure 2: Recognizing people in image collections.

## Extracting Embeddings

We start by locating faces and upper bodies of people visible in a given image. Faces are frequently occluded or simply not visible if the subject is looking away from the camera. To solve these cases we also consider the upper bodies of the people in the image, since they usually show constant characteristics—like clothing—within a specific context. These constant characteristics can provide strong cues to identify the person across images captures a few minutes from each other.

We rely on a deep neural network that takes a full image as input, and outputs bounding boxes for the detected faces and upper bodies. We then associate face bounding boxes with their corresponding upper bodies by using a matching routine that takes into account bounding box area and position, as well as the intersection of the face and upper body regions.

The face and upper body crops obtained from an image are fed to a pair of separate deep neural networks whose role is to extract the feature vectors, or embeddings, that represent them. Embeddings extracted from different crops of the same person are close to each other and far from embeddings that come from crops of a different person. We repeat this process of detecting face and upper body bounding boxes and extracting the corresponding feature vectors on all assets in a Photos library. This repetition results in a collection of face and upper body embeddings.

## Building a Gallery

A gallery is a collection of frequently occurring people in a user's Photos library. To build the gallery in an unsupervised manner, Photos relies on clustering techniques to form groups, or clusters, of face and upper body feature vectors that correspond to the people detected in the library. In Photos, we developed a novel agglomerative clustering algorithm that enables an efficient incremental update of existing clusters and can scale to large libraries. To build these clusters, we start with a clustering

algorithm that uses a combination of the face and upper body embeddings for each observation. This step is fairly conservative because when it joins two instances, they are permanently associated. We tune the algorithm so that each first-pass cluster only groups together very close matches, providing high precision but many, smaller clusters. Each cluster is represented by the running average of its embeddings as instances are added.

Over time, upper body embeddings are less robust than face embeddings because they rely on a person's temporary appearance—for example their clothing on a specific day. Accordingly, during this first pass, we're careful to compare upper body embeddings only from the same moment. A moment links together a collection of assets based on metadata such as time and location. Within the assets of a given moment, we expect a person's upper body to have a consistent appearance. For example, the person is most likely wearing the same clothes. There are many ways to approach useful combinations of observations, such as learning to combine the face and upper body embeddings into a single vector, or creating a concatenation of the two embeddings. The solution we settled on employs distances computed in each embedding space separately, using the formula  $D_{ij} = \min(F_{ij}, \alpha \cdot F_{ij} + \beta \cdot T_{ij})$  where  $D_{ij}$  is the distance between two observations,  $F$  represents the face embedding distance, and  $T$  represents the upper body embedding distance. When only the face embedding is available in an observation, we compare it with the mean face embedding of the identified clusters. If the upper body embedding is also available and its distance to an existing cluster from the same moment is helpful, we use a linear combination of the face and the upper body distance. Finally, if only the upper body embedding is present we simply use the upper body distance. We've carefully tuned the set of face and upper body distance thresholds to get the most out of the upper body embedding without negatively impacting overall accuracy.

After the first pass of clustering using the greedy method, we perform a second pass using **hierarchical** agglomerative clustering (HAC) to grow the clusters further, increasing recall significantly. The second pass uses only face embedding matching, to form groups across moment boundaries. The **hierarchical** algorithm recursively merges pairs of clusters that minimally increase the linkage distance. Our linkage strategy uses the median distance between the members of two HAC clusters, and then switches to a random sampling method when the number of comparisons gets significant. Thanks to a few algorithmic optimizations, this method has runtime and memory performance characteristics similar to single-linkage HAC, but has accuracy characteristics on par or better than average-linkage HAC.

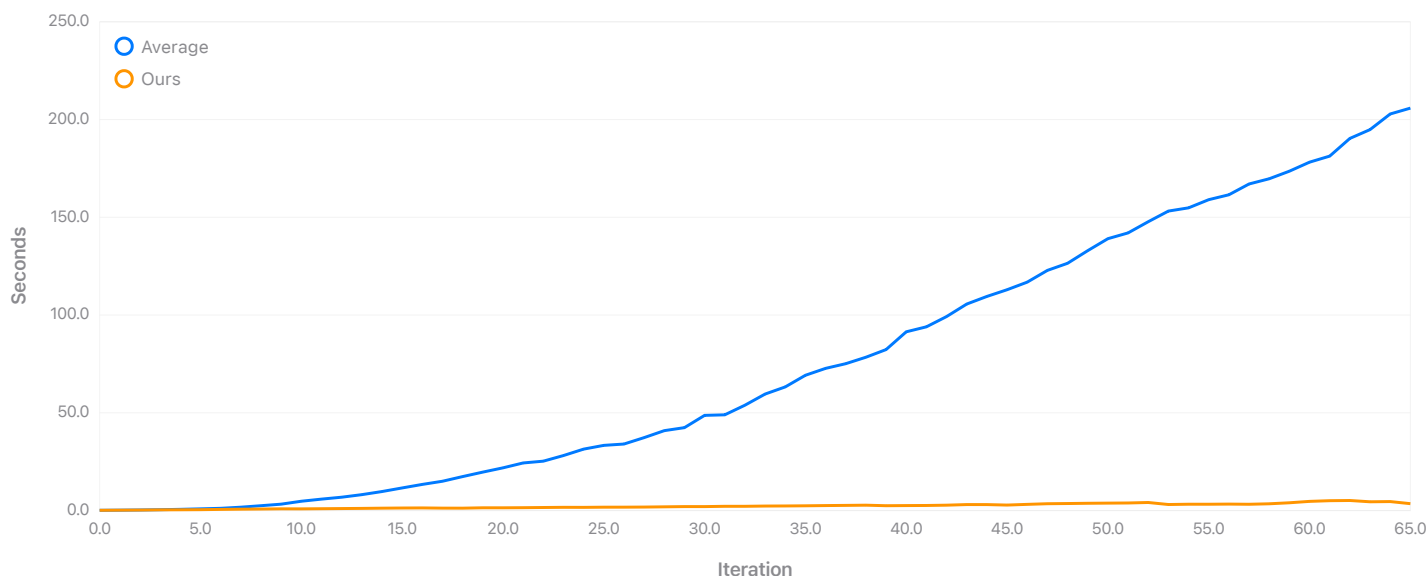


Figure 3: Processing time per iteration for typical average linkage clustering compared to our approach (lower is better).

This clustering algorithm runs periodically, typically overnight during device charging, and assigns every observed person instance to a cluster. If the face and upper body embeddings are well trained, the set of the  $K$  largest clusters is likely to correspond to  $K$  different individuals in a library. Using a number of heuristics based on the distribution of cluster sizes, inter- and intra-cluster distances, and explicit user input via the app, Photos determines which set of clusters together comprises the gallery of known individuals for a given library.

## Assigning Identity

The second phase in the person recognition problem is to match a new observation to this gallery. Inspired by [Learning Feature Representations with K-means](#), we choose an encoder more powerful than the naive one-hot encoding of nearest neighbor classification. Specifically, we represent each cluster with a set of canonical exemplars  $X_0, X_1, \dots, X_c$  instead of just using the centroid. The gallery is then represented by a dictionary  $D$  comprised of all the exemplars,  $D = [X_0^1, X_1^1, \dots, X_c^1, X_0^2, X_1^2, \dots, X_c^2, \dots, X_0^K, X_1^K, \dots, X_c^K]$  where the superscript represents the  $K$ -th person in the gallery. Now, given a new observation,  $y$ , we solve the sparse coding problem  $\min_x \|y - D \cdot x\|_2^2 + \lambda \cdot \|x\|_1$ .

The final assignment of  $y$  is made to the cluster corresponding to the maximum total “energy” of the sparse code,  $x$ . This generalization over nearest neighbor classification provides better accuracy, particularly in two regimes: when the size of each cluster is relatively small, and when more than one cluster in the gallery could belong to the same identity. Photos uses this technique to quickly identify people as someone captures photographs. This enables Photos to adapt more dynamically to user’s libraries as new observations become available.

## Filtering Unclear Faces

The processing pipeline we've described so far would assign every computed face and upper body embedding to a cluster during overnight clustering. However, not every observation corresponds to real faces and upper bodies, and not all faces and upper bodies can be well represented by a neural network running on a mobile device. Over time, face and upper body detections that are either false positives or out-of-distribution would start appearing in the gallery and start impacting recognition accuracy. To combat this, an important aspect of the processing pipeline is to filter out such observations that are not well represented as face and upper body embedding.

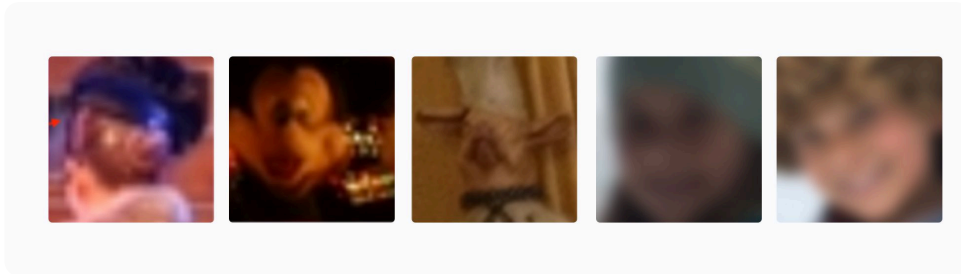


Figure 4: Examples of false positive and unclear faces.

## Fairness in the Generation of Face Embeddings

A challenge in obtaining a useful face representation is ensuring consistent accuracy on many axes. The model must show similar performance across various age groups, genders, ethnicity, skin tones and other attributes. Fairness is an essential aspect of model development and must be taken into account from the beginning, not only in the data collection which needs to be diverse, inclusive, and balanced but also in strong failure analysis and model evaluation.

### Data and Augmentation

We're constantly improving the variety in our datasets while also monitoring for bias across axes mentioned before. Awareness of biases in the data guides subsequent rounds of data collections and informs model training. Some of the most effective datasets we have curated use a paid crowd-sourced model with a managed crowd to gather representative image content from participants across the globe spanning various age groups, genders, and ethnicities.

Major improvements to model accuracy can also come from data augmentation. During training we use a random combination of many transformations to augment the input image in order to improve model generalization. These transformations include pixel-level changes such as color jitter or grayscale conversion, structural changes like left-right flipping or distortion, Gaussian blur, random compression artifacts and [cutout regularization](#). As learning progresses, the transformations get added incrementally in a curriculum-learning fashion. The model initially learns to differentiate between easier examples and, as training goes on, is taught harder examples.

For example, to tackle the specific issue of the proliferation of face masks to combat the COVID-19 pandemic, we designed a synthetic mask augmentation. We used face landmarks to generate a realistic shape corresponding to a face mask. We then overlaid random samples from clothing and other textures in the inferred mask area



over the input face. These synthetic masks allow the model to give more importance to other areas of the face and generalize better when a mask is present while not impacting accuracy for non-masked faces.

## Designing the Network

The main challenge in designing the architecture is capturing the highest accuracy possible while running efficiently on-device, with low latency and a thin memory profile. There are trade-offs at every stage of the network that require experimentation to balance accuracy and computational cost. We settled on a deep neural network structure inspired by the lightweight and efficient model proposed in [AirFace](#) [↗](#). We optimized the blocks for the task at hand and significantly increased the network depth.

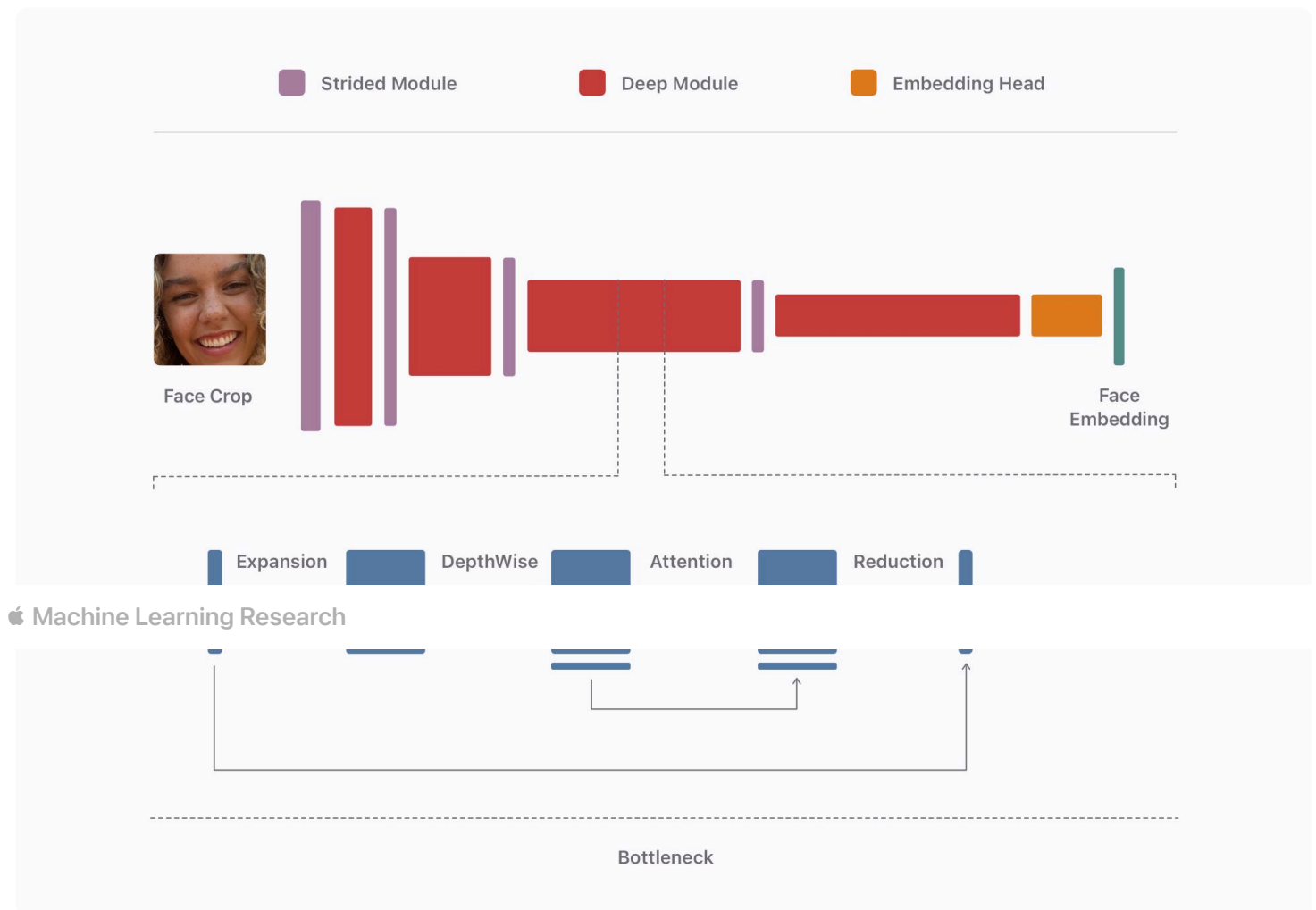


Figure 5: Face embedding network architecture.

As shown in [Figure 5](#), the backbone is composed of alternating shallow strided modules that we use to reduce the dimensionality of the feature map and increase the depth of the network, and along with deep modules containing multiple bottlenecks as inspired by [MobileNetv3](#) [↗](#), internally providing a higher-dimensional feature space while keeping a compact representation between blocks.

Each bottleneck follows an inverted residual and linear structure with a lightweight attention layer. It consists of a point-wise expansion convolution with a tuned per-

level ratio to expand the number of channels, followed by a spatial depth-wise convolution. We apply a channel attention block inspired by [Squeeze and Excitation](#) to this largest representation. We then use a second point-wise reduction convolution as a projection layer to reduce the number of channels and, finally, we connect input and output through a residual connection if they have the same number of channels. Inside the bottlenecks, we use non-linear activations and batch normalization.

To convert the final feature map of our network into our embedding, we use a linear global depth-wise convolution as proposed in the mobile recognition network [MobileFaceNet](#). This is a better solution than typical pooling mechanisms as it lets the network learn and focus on the relevant parts of the receptive field, which is integral for recognizing faces. We then normalize this embedding to create the face embedding.

## Training the model

The goal during training is to obtain an embedding which promotes intra-class compactness and inter-class discrepancy on the unit-hypersphere. The lower part of [Figure 6](#) shows this concept: before training the network, the embeddings are randomly distributed across the hypersphere. As training progresses, embeddings representing faces from the same person get closer together and further from embeddings representing another person. This procedure is similar to that described in [ArcFace](#), a state-of-the-art recognition method.

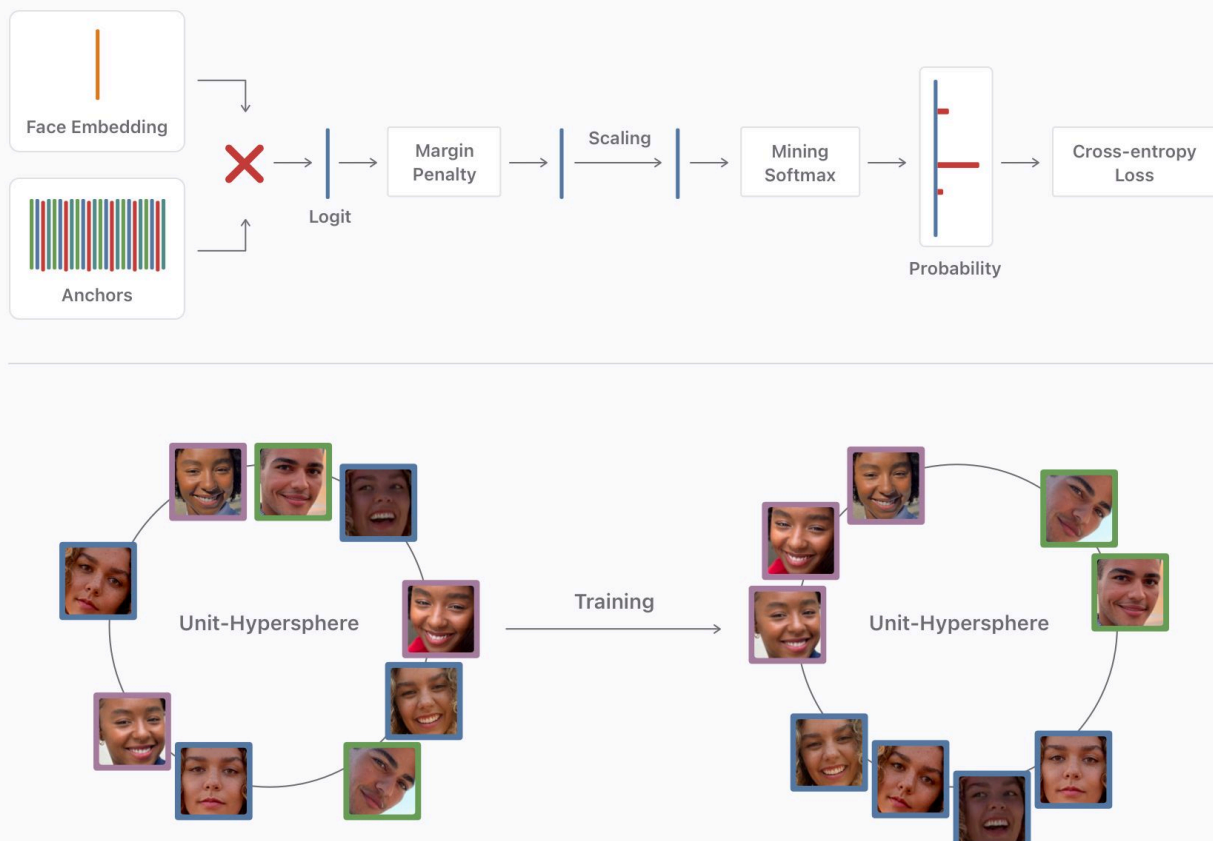


Figure 6: Model training procedure and distribution of the face embeddings before and after training.



We compare the face embedding extracted by the deep convolutional neural network described previously, to a set of anchors. The first step is to compute the dot-product between the normalized embedding and each anchor to obtain their cosine similarity. These anchors are a trained set of parameters that can be interpreted as the center of the face embeddings representing each person in our training dataset. The result is a matrix of size  $N \times I$  where  $I$  is the number of identities and  $N$  matches the size of the face embedding. The cosine similarity for the positive anchor  $\cos(\theta_{y_i})$  is then modified by adding an angular margin penalty  $m$  to increase the distance between different classes. This additive margin significantly improves the discriminative power of our embedding. Finally, we rescale the obtained features by  $s$  and use it as logit to compute the softmax cross-entropy loss based on the equation below.

$$L_i = -\log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i}^K e^{s f(\cos \theta_j)}}$$

We integrate the concept of mining into the softmax cross-entropy loss by applying a strategy similar to the [Support Vector Guided Softmax](#) and the adaptive curriculum learning loss introduced in [CurricularFace](#). This allows us to underweight easy examples and give more importance to the hard ones directly in the loss. We use re-weighting function  $f$  to modulate the similarity  $\cos(\theta_j)$  for the negative anchors proportionally to their difficulty. This margin-mining softmax approach has a significant impact on final model accuracy by preventing the loss from being overwhelmed by a large number of easy examples. The additive angular margin loss can present convergence issues with modern smaller networks and often can only be used in a fine tuning step. This problem does not appear when using our approach and the model easily converges when trained from random initialization.

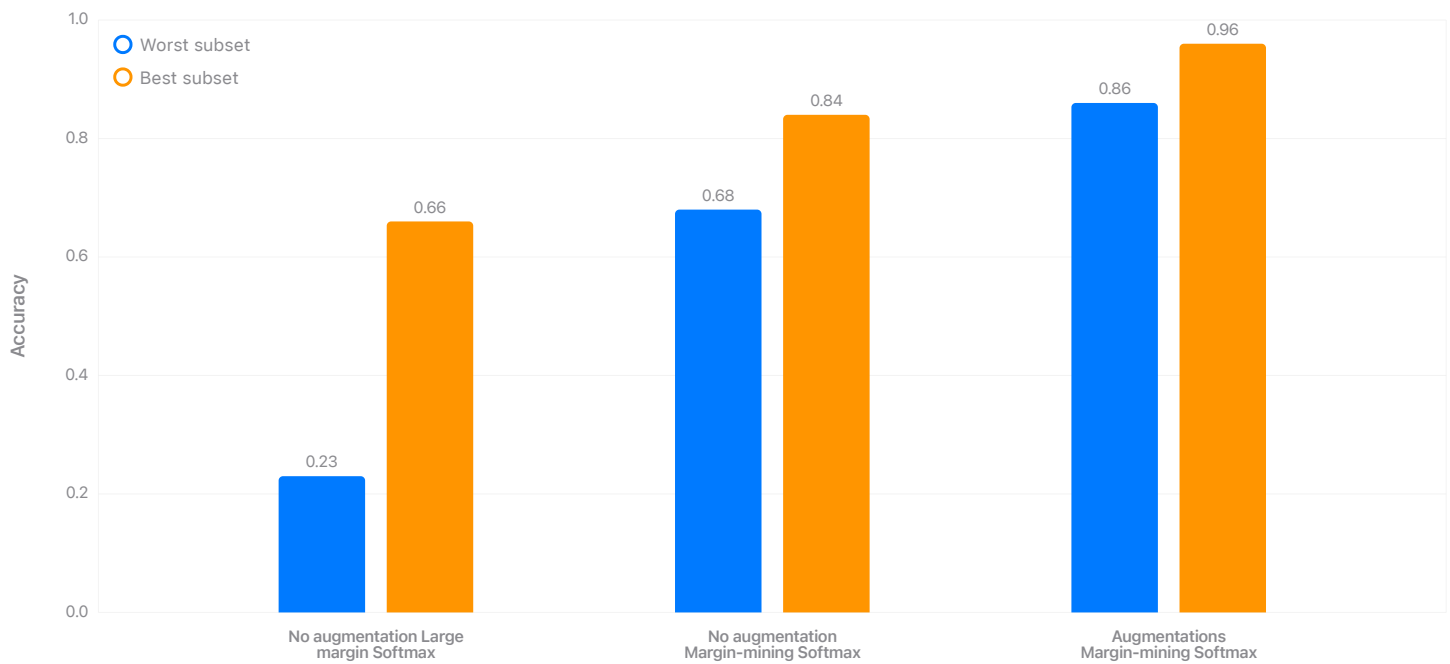


Figure 7: Impact of training procedure on model accuracy.

The impact of the different aspects of the training we discussed is visible in [Figure 7](#). For each set of parameters we show the accuracy on the worst and best performing subsets of a large and diverse dataset. We can see not only that the final method significantly improves accuracy but also that it helps bridge the gap between sub-groups.

We train this neural network from random initialization using the adaptive gradient algorithm [AdamW](#), which decouples weight decay from the gradient update. The main learning rate is carefully tuned and follows a schedule based on the [One Cycle Policy](#).

Finally, to quantify uncertainty and detect out-of-distribution samples, we fine-tune an ad-hoc embedding confidence branch. We use random, non-face crop, as well as out-of-domain data, to help the uncertainty branch detect anomalies in an input image that would render the embedding less relevant. This is an important aspect of modeling: modern neural networks can be overconfident when seeing samples far from the training distribution.

## On-Device Performance

On-device performance is especially important as the end-to-end process runs entirely locally, on the users device, keeping the recognition processing private.

To get the best performance and inference latency while minimizing memory footprint and power consumption our model runs end-to-end on the Apple Neural Engine (ANE). On recent iOS hardware, face embedding generation completes in less than 4ms. This gives an 8x improvement over an equivalent model running on GPU, making it available to real-time use cases.

## Visualizing Results

This latest advancement, available in Photos running iOS 15, significantly improves person recognition. As shown in [Figure 8](#), using private, on-device machine learning we can correctly recognize people with extreme poses, accessories, or even occluded faces and use the combination of face and upper body to match people whose faces are not visible at all. This significantly improves the Photos experience by identifying the people who matter most to us in situations where it was previously impossible.



Figure 8: Person recognition using the new architecture in Photos.

## Acknowledgments

Many people contributed to this research, including Floris Chabert, Jingwen Zhu, Brett Keating, and Vinay Sharma.

## References

Adam Coates, Andrew Y. Ng. Learning Feature Representations with K-means. In Neural Networks: Tricks of the Trade, 2012. [\[link ↗\]](#).

Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, Hartwig Adam. Searching for MobileNetV3. arXiv:1905.02244, May, 2019. [\[link ↗\]](#).

Ilya Loshchilov, Frank Hutter. Decoupled Weight Decay Regularization. arXiv:1711.05101, November, 2017. [\[link ↗\]](#).

Jiankang Deng, Jia Guo, Niannan Xue, Stefanos Zafeiriou. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. arXiv:1801.07698, January, 2018. [\[link ↗\]](#).

Jie Hu, Li Shen, Samuel Albanie, Gang Sun, Enhua Wu. Squeeze-and-Excitation Networks. arXiv:1709.01507, September, 2017. [\[link ↗\]](#).

Leslie N. Smith. A disciplined approach to neural network hyper-parameters. arXiv:1803.09820, March, 2018. [\[link ↗\]](#).

Sheng Chen, Yang Liu, Xiang Gao, Zhen Han. MobileFaceNets: Efficient CNNs for Accurate Real- Time Face Verification on Mobile Devices. arXiv:1804.07573, April, 2018. [\[link ↗\]](#).

Terrance DeVries, Graham W. Taylor. Improved Regularization of Convolutional Neural Networks with Cutout. arXiv:1708.04552, November, 2017. [\[link ↗\]](#).

Yuge Huang, Yuhan Wang, Ying Tai, Xiaoming Liu, Pengcheng Shen, Shaoxin Li, Jilin Li, Feiyue Huang. CurricularFace: Adaptive Curriculum Learning Loss for Deep Face Recognition. arXiv:2004.00288, April, 2020. [\[link ↗\]](#).

Xianyang Li, Feng Wang, Qinghao Hu, Cong Leng. AirFace: Lightweight and Efficient Model for Face Recognition. arXiv:1907.12256, July, 2019. [\[link ↗\]](#).

Xiaobo Wang, Shuo Wang, Shifeng Zhang, Tianyu Fu, Hailin Shi, Tao Mei. Support Vector Guided Softmax Loss for Face Recognition. arXiv:1812.11317, December, 2018. [\[link ↗\]](#).

## Related readings and updates.

---

June 7, 2022 | [Computer Vision](#), [Methods and Algorithms](#)

### A Multi-Task Neural Architecture for On-Device Scene Analysis

---

Scene analysis is an integral core technology that powers many features and experiences in the Apple ecosystem. From visual content search to powerful memories marking special occasions in one's life, outputs (or "signals") produced by scene analysis are critical to how users interface with the photos on their devices. Deploying dedicated models for each of these individual features is inefficient as many of these models can benefit from

October 19, 2021 | [Computer Vision](#), [Methods and Algorithms](#)

### On-device Panoptic Segmentation for Camera Using Transformers

---

Camera (in iOS and iPadOS) relies on a wide range of scene-understanding technologies to develop images. In particular, pixel-level understanding of image content, also known as *image segmentation*, is behind many of the app's front-and-center features. Person segmentation and depth estimation powers Portrait Mode, which simulates effects like the shallow depth of field and Stage Light. Person and skin segmentation power semantic

sharing resources. We present how we developed Apple Neural Scene Analyzer (ANSA), a unified backbone to build and maintain scene analysis workflows in production. This was an important step towards enabling Apple to be among the first in the industry to deploy fully client-side scene analysis in 2016.

[Read more](#)

rendering in group shots of up to four people, optimizing contrast, lighting, and even skin tones for each subject individually. Person, skin, and sky segmentation power Photographic Styles, which creates a personal look for your photos by selectively applying adjustments to the right areas guided by segmentation masks, while preserving skin tones. Sky segmentation and skin segmentation power denoising and sharpening algorithms for better image quality in low-texture regions. Several other features consume image segmentation as an essential input.

[Read more](#)

## Discover opportunities in Machine Learning.

Our research in machine learning breaks new ground every day.

[Work with us](#)

[Machine Learning Research](#) [Publications](#) [Recognizing People in Photos Through Private On-Device Machine Learning](#)

[Privacy Policy](#) [Terms of Use](#) [Legal](#)

Copyright © 2025 Apple Inc. All rights reserved.